

Architecture des Ordinateurs — Chapitres 2 à 5

Nom : BARETS

Prénom : Emma

ID Étudiant : 100672190DG

Table des matières

Chapitre 2 — La Représentation des Nombres Entiers Positifs	3
Chapitre 3 — Opérations en Binaire & Représentation des Nombres Signés	13
Chapitre 4 — Représentation des Nombres Flottants et des Caractères	19
Chapitre 5 - Algèbre de Boole	28

Chapitre 2 — La Représentation des Nombres Entiers Positifs

2.6 Exercices

Pour ce chapitre 2, vous devez rendre les exercices **2.3**, **2.12** et deux exercices "**Au Choix**".

L'exercice **2.2**, qui vous demandera un peu de programmation, est pour un **Bonus**.

Exercice 2.1 (Au Choix)

Convertir 31 et 96 en chiffres romains et effectuer l'addition dans cette représentation.

Convertir le résultat de cette addition dans la base 10 usuelle et vérifier qu'il est bien égal à 127.

Applicatif du cours sans difficultés particulières.

Réponse : 31 en chiffre romain est XXXI, 96 en chiffre romain est XCVI. Leur addition donne CXXVII, ce qui correspond à 127 en base 10 ($31 + 96 = 127$).

Exercice 2.2 (Bonus)

Décrire dans le détail, avec des phrases, la suite d'opérations qu'il faut effectuer pour additionner deux nombres écrits en chiffres romains. Écrire ensuite le programme correspondant dans le langage de votre choix (cf. Petit Guide).

Pour vous aider : Une manière "intelligemment-paresseuse" de procéder est d'effectuer l'addition en base dix des nombres saisis par l'utilisateur puis de convertir le résultat à l'aide du programme réalisé dans l'exercice 2.3.

Réponse :

Pour additionner deux nombres écrits en chiffres romains il faut d'abord demander à l'utilisateur d'entrer deux nombres en chiffres romains, par exemple X et V. Ensuite on va convertir chaque nombre romain en nombre entier à l'aide d'une fonction qui va associer chaque symbole romain à sa valeur et applique les règles de soustraction. Puis on additionne les deux entiers obtenus pour obtenir la somme en base 10. On convertit cette somme en chiffre romain (voir programme ex 2.3) et on affiche le résultat en chiffre romain.

(PS : J'ai également découvert ".upper()" dans ce cours mais je ne sais pas si on peut l'utiliser donc j'ai préféré inclure dans "valeurs" les minuscules et les majuscules : <https://fr.python-3.com/?p=2788>)

Pour insérer du code Python dans ce fichier LaTeX, j'ai utilisé ce site pour comprendre comment faire : <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>.

```

1 # Exercice 2.2 (Bonus)
2
3 def romain_vers_entier(romain):
4     """
5     Convertit un chiffre romain en entier.
6     Prend en compte les majuscules et les minuscules.
7     """
8     # Dictionnaire avec la valeur de chaque symbole romain
9     valeurs = {
10         'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000,
11         'i':1, 'v':5, 'x':10, 'l':50, 'c':100, 'd':500, 'm':1000
12     }
13
14     total = 0
15     i = 0
16
17     while i < len(romain):
18         lettre = romain[i]
19         if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
20             total += valeurs[romain[i + 1]] - valeurs[lettre]
21             i += 2
22         else:
23             total += valeurs[lettre]
24             i += 1
25
26     return total
27
28 romain1 = input("Entrez le premier nombre romain : ")
29 romain2 = input("Entrez le second nombre romain : ")
30
31 entier1 = romain_vers_entier(romain1)
32 entier2 = romain_vers_entier(romain2)
33 somme = entier1 + entier2
34
35 print(f"La somme en base 10 est : {somme}")

```

Quelques tests réalisés pour voir si le programme fonctionne bien :

Calcul de base :

```

>>> %Run 2_2_3.py
Entrez le premier nombre romain : V
Entrez le second nombre romain : V
La somme en base 10 est : 10

```

FIGURE 1 – Test 1

Si un symbole plus petit est avant le plus grand + test avec les minuscules

```
>>> %Run 2_2_3.py
Entrez le premier nombre romain : iv
Entrez le second nombre romain : v
La somme en base 10 est : 9
```

FIGURE 2 – Test 2

Voici une deuxième version du programme (Voir explication et test¹)

```
1 # Exercice 2.2 (Bonus)
2
3 def romain_vers_entier(romain):
4     """
5     Convertit un chiffre romain en entier.
6     Gere les majuscules et minuscules.
7     Applique les regles classiques :
8     - Pas plus de 3 repetitions consecutives pour I, X, C, M
9     - V, L, D ne sont jamais repetes
10    - Limite de 3999 pour le resultat
11    """
12
13    # Dictionnaire contenant la valeur de chaque symbole romain
14    valeurs = {
15        'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000,
16        'i':1, 'v':5, 'x':10, 'l':50, 'c':100, 'd':500, 'm':1000
17    }
18
19    # Lettres autorisees a etre repetees jusqu'a 3 fois
20    repetables = {'I', 'X', 'C', 'M', 'i', 'x', 'c', 'm'}
21
22    compteur = 1 # Compteur pour suivre le nombre de repetitions d'affilees
23    total = 0    # Somme finale en base 10
24    i = 0        # Index pour parcourir la chaine
25
26    # Parcours de chaque symbole du nombre romain
27    while i < len(romain):
28        lettre = romain[i]
29
30        # Verification que le caractere est bien un symbole romain
31        if lettre not in valeurs:
32            print(f"Symbole romain invalide detecte : {lettre}")
33            return None
34
35        # Verification des repetitions d'affilees
36        if i > 0 and romain[i] == romain[i-1]:
37            compteur += 1
38            # Si le symbole peut etre repete, on limite a 3
39            if lettre in repetables and compteur > 3:
40                print(f"Erreur : le symbole {lettre} ne peut pas etre repete plus de
41                    3 fois consecutivement.")
42                return None
43            # Si le symbole ne peut pas etre repete du tout (V, L, D)
44            elif lettre not in repetables:
```

```

44         print(f"Erreur : le symbole {lettre} ne peut pas etre repete.")
45         return None
46     else:
47         compteur = 1 # Reinitialisation du compteur si symbole different
48
49     # Verification que le total ne depasse pas 3999
50     if total > 3999:
51         print("Attention : la numeration romaine standard ne supporte pas les
52             nombres > 3999")
53         return None
54
55     # Dans le cas ou le premier nombre est plus petit que le deuxieme
56     if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
57         total += valeurs[romain[i + 1]] - valeurs[lettre]
58         i += 2 # On saute deux symboles car ils ont ete utilises ensemble
59     else:
60         total += valeurs[lettre]
61         i += 1 # On passe au symbole suivant
62
63     return total # Renvoie la valeur entiere obtenue

```

Exercice 2.3 (A Rendre)

Réaliser un programme, dans le langage de votre choix (cf Petit Guide), qui imprime un nombre saisi par l'utilisateur en chiffres romains.

Pour vous aider : Voir les liens de la section "Pour aller (Encore) plus loin".

```
1 # Exercice 2.3 (A Rendre)
2
3 def entier_vers_roman(nombre):
4     """
5     Convertit un entier en chiffre romain.
6     """
7     # Liste des symboles romains et leur valeur associee (du plus grand au plus petit
8     # pour la conversion)
9     romains = [
10         ('M',1000), ('CM',900), ('D',500), ('CD',400),
11         ('C',100), ('XC',90), ('L',50), ('XL',40),
12         ('X',10), ('IX',9), ('V',5), ('IV',4), ('I',1)
13     ]
14
15     resultat = "" # Chaine pour la conversion finale
16
17     # Parcours de chaque symbole et sa valeur
18     for symbole, valeur in romains:
19         # Tant que le nombre restant est superieur ou egal a la valeur du symbole
20         while nombre >= valeur:
21             resultat += symbole # Ajouter le symbole romain au resultat
22             nombre -= valeur    # Soustraire cette valeur du nombre restant
23
24     return resultat # Resultat du chiffre romain
25
26 # Conversion de la somme obtenue avant en chiffre romain
27 romain_resultat = entier_vers_roman(somme)
28
29 # Affiche la somme des deux nombres saisis en chiffres romains
30 print(f"La somme de {romain1} et {romain2} en chiffres romains est : {romain_resultat}")
```

Pour tester les programmes précédents, j'ai utilisé le site que vous nous avez conseillé dans "Pour aller plus loin" (WolframAlpha) : <https://www.wolframalpha.com/>

Exemple de test avec ce site :



FIGURE 3 – Test 1

```
>>> %Run 2_2_3.py

Entrez le premier nombre romain : XLVIII
Entrez le second nombre romain : LXXII
La somme en base 10 est : 120
```

FIGURE 4 – Test 2

Test pour voir si les deux programmes fonctionnent ensemble :

```
>>> %Run 2_2_3.py

Entrez le premier nombre romain : L
Entrez le second nombre romain : C
La somme en base 10 est : 150
La somme de L et C en chiffres romains est : CL
```

FIGURE 5 – Test 2

Explication : De plus, en lisant la page Wikipedia (le lien que vous nous avez fourni pour aller plus loin : https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine) je pense qu'on aurait aussi pu rajouter et modifier légèrement le programme, c'est pour cela que j'ai décidé de faire une v2.

On apprend que le système romain ne gère pas naturellement les fractions ou les nombres <1 , et que tout ce qu'on fait en chiffres romains concerne des entiers ≥ 1 (cela n'impactera pas ces deux programmes).

La numération romaine traditionnelle permet de représenter les nombres jusqu'à 3999. Par convention, le chiffre 4 s'écrit IV (5-1), 40 s'écrit XL et 400 s'écrit CD. Il n'existe pas de symbole officiel pour 5000, ce qui fixe une limite pratique pour cette numération. Cette limite est mentionnée par Georges Ifrah dans *Histoire universelle des chiffres* (tome 1, page 454).

Certaines discussions suggèrent que des variantes existaient : par exemple, des répétitions plus longues de symboles comme IIII pour 4 ou XXXX pour 40 ont pu être utilisées, mais il n'y a pas de preuve formelle que ce soit systématique.

Donc on aurait pu aussi rajouter une condition pour que les III s'arrête à 3 ou à 4.

Bien que certaines sources montrent qu'il serait possible de gérer des nombres supérieurs à 3999 ou d'utiliser des variations historiques, ces cas restent majoritairement des suppositions ou des exceptions. Pour cette raison, j'ai décidé de rajouter une condition pour que mon programme se limite aux nombres ≤ 3999 et également la règle que les lettres I, X, C et M ne peuvent pas apparaître plus de trois fois consécutivement, et que V, L et D ne peuvent jamais se répéter, afin de respecter les conventions classiques de la numération romaine.

Voici la V2 de la question 2.3 :

```
1 # Exercice 2.3 (A rendre)
2
3 def entier_vers_romain(nombre):
4
5     if nombre > 3999:
6         print("La numeration romaine standard ne supporte pas les nombres > 3999")
7         return None
8
9     # Liste des symboles romains et leur valeur associee (du plus grand au plus petit
10    )
11    romains = [
12        ('M',1000), ('CM',900), ('D',500), ('CD',400),
13        ('C',100), ('XC',90), ('L',50), ('XL',40),
14        ('X',10), ('IX',9), ('V',5), ('IV',4), ('I',1)
15    ]
16
17    resultat = "" # Chaine pour la conversion finale
18
19    # Parcours de chaque symbole et de sa valeur
20    for symbole, valeur in romains:
21        count = 0 # Compteur pour limiter les repetitions
22
23        # Boucle tant que le nombre reste >= valeur et que la repetition est
24        autorisee
25        while nombre >= valeur and (
26            (symbole in ['I','X','C','M'] and count < 3) or
27            (symbole in ['V','L','D'] and count < 1) or
28            (symbole in ['CM','CD','XC','XL','IX','IV']) # symboles composes sans
29            limite
30        ):
31            resultat += symbole
32            nombre -= valeur
33            count += 1
34
35    return resultat
```

```

29         nombre -= valeur
30         count += 1
31
32     return resultat # Retourne le chiffre romain
33
34 # Conversion de la somme en chiffre romain
35 romain_resultat = entier_vers_roman(somme)
36
37 # Affichage du resultat ou message d'impossibilite
38 if romain_resultat is not None:
39     print(f"La somme de {romain1} et {romain2} en chiffres romains est : {
        romain_resultat}")
40 else:
41     print("Impossible de convertir ce nombre en chiffres romains standard.")

```

Pour les programmes finaux des exercices 2.2 et 2.3, j'ai inclus un fichier Python en annexe afin que vous puissiez les exécuter et les tester directement.

```

Entrez le premier nombre romain : IIII
Entrez le second nombre romain : I
Erreur : le symbole I ne peut pas être répété plus de 3 fois consécutivement.
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.

```

FIGURE 6 – Test 1 : Test en mettant plusieurs lettres d'affilées

```

Entrez le premier nombre romain : XXXXXXMMXXMMXXMMXXMMXXMMXXMMXXMMXXMMXXMMXX
Entrez le second nombre romain : X
Attention : la numération romaine standard ne supporte pas les nombres > 3999
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.

```

FIGURE 7 – Test 2 : Test en obtenant un nombre supérieur à 3999

```

Entrez le premier nombre romain : VV
Entrez le second nombre romain : V
Erreur : le symbole V ne peut pas être répété.
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.

```

FIGURE 8 – Test 3 : Test répétition lettre V

Exercice 2.9 (Au Choix)

Convertir le nombre $(1254)_7$ de la base 7 vers le décimal

Réponse :

Le nombre 1254 en base 7 a 4 chiffres :

- 4 est à la position 0 (7^0)
- 5 est à la position 1 (7^1)
- 2 est à la position 2 (7^2)
- 1 est à la position 3 (7^3)

On multiplie chaque chiffre par sa puissance de 7 correspondante :

- Chiffre le plus à gauche : $1 \cdot 7^3 = 1 \cdot 343 = 343$
- Chiffre suivant : $2 \cdot 7^2 = 2 \cdot 49 = 98$
- Chiffre suivant : $5 \cdot 7^1 = 5 \cdot 7 = 35$
- Chiffre le plus à droite : $4 \cdot 7^0 = 4 \cdot 1 = 4$

On les additionne ensuite ce qui nous donne : $343 + 98 + 35 + 4 = 480$

1254 en base 7 est donc 480 en base 10.

Exercice 2.12 (A Rendre)

Convertir le nombre $(3391)_{10}$ en base -5. (Facile si vous avez déjà répondu au 2.11)

Rappel : les restes des divisions par -5 doivent être positifs. Vous ne devez pas utiliser la forme (par exemple) $18 = 3 \cdot 5 + 3$ mais plutôt $18 = 4 \cdot 5 + 2$.

Réponse :

Pour convertir le nombre 3391 en base -5, on fait :

1) 3391 divisé par -5 = -678 avec un reste de 1.

On écrit donc : $3391 = (-678) \cdot (-5) + 1$

2) -678 divisé par -5 = 136 avec un reste de 2.

On écrit : $-678 = 136 \cdot (-5) + 2$

3) 136 divisé par -5 = -27 avec un reste de 1.

On écrit : $136 = (-27) \cdot (-5) + 1$

4) -27 divisé par -5 = 6 avec un reste de 3.

On écrit : $-27 = 6 \cdot (-5) + 3$

5) 6 divisé par -5 = -1 avec un reste de 1.

On écrit : $6 = (-1) \cdot (-5) + 1$

6) -1 divisé par -5 = 0 avec un reste de -1.

Mais on veut un reste positif donc on ajuste : reste = 4 et quotient = 1.

On écrit : $-1 = 1 \cdot (-5) + 4$

Ensuite, on lit tous les restes de bas en haut pour obtenir le nombre en base -5.

Donc, $(3391)_{10} = (141312)_{-5}$.

Sources

- Python-3.com — Explication sur la méthode `.upper()` : <https://fr.python-3.com/?p=2788>
- BornToCode.fr — Comment insérer et styliser du code Python en LaTeX : <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>
- WolframAlpha — Outil de vérification des calculs : <https://www.wolframalpha.com/>
- Wikipédia — Discussion sur la numération romaine : https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine
- Georges Ifrah, *Histoire universelle des chiffres*, tome 1, Robert Laffont, 1998, p. 454.

Chapitre 3 — Opérations en Binaire & Représentation des Nombres Signés

3.5 Une première Série d'Exercices

Exercice 3.6 (A Rendre)

La base -2 n'utilise que les chiffres 0 et 1. (Facile) Traduire en décimal les seize valeurs entre $(0000)_{-2}$ et $(1111)_{-2}$. (Plus difficile) Quelle est la table d'addition de la base -2 ?

Pour vous aider : voir l'article "Système négabinaire" sur Wikipédia...

Réponse :

Selon le cours, pour convertir un nombre écrit en base négative (ici base -2) en décimal, on applique la même méthode que pour toute base : chaque chiffre (ou bit) est multiplié par la puissance correspondante de la base, en commençant par la droite avec la puissance 0. La valeur décimale du nombre est ensuite la somme de toutes ces valeurs.

Pour la base -2, le bit le plus à droite correspond à $(-2)^0 = 1$, le suivant à $(-2)^1 = -2$, puis $(-2)^2 = 4$, et enfin $(-2)^3 = -8$. En appliquant cette logique, on peut calculer les valeurs décimales de tous les nombres à 4 bits.

Par exemple pour 1001_{-2} : $1 \cdot (-2)^3 + 0 \cdot (-2)^2 + 0 \cdot (-2)^1 + 1 \cdot (-2)^0 = -7$

On fait cela de 0000_{-2} à 1111_{-2} :

Base -2	Décimal
0000	0
0001	1
0010	-2
0011	-1
0100	4
0101	5
0110	2
0111	3
1000	-8
1001	-7
1010	-10
1011	-9
1100	-4
1101	-3
1110	-6
1111	-5

Pour construire la table d'addition en base -2, il faut comprendre que chaque bit ne peut prendre que deux valeurs : 0 ou 1. On fait l'addition bit à bit comme en binaire classique, mais on tient compte de la retenue négative.

$A + B$	Résultat	Retenue
$0 + 0$	0	0
$0 + 1$	1	0
$1 + 0$	1	0
$1 + 1$	0	-1

$1 + 1 = 0$ avec retenue -1 (car $2 = 0 + (-2) \cdot (-1)$). Les autres additions ($0+0$, $0+1$, $1+0$) restent simples, retenue = 0.

3.10 Une Seconde Série d'Exercices...

Pour ce chapitre trois, vous devez rendre les exercices **3.6**, **3.14** et un exercice "**Au Choix**". L'exercice **3.15** est pour un Bonus.

Exercice 3.8 (Au Choix)

Avec quatre bits, nous pouvons représenter 16 valeurs différentes; énumérez ces 16 valeurs et indiquez pour chacune d'entre elles le nombre qu'elle code en **complément à 2**.

Réponse :

On veut énumérer toutes les valeurs à 4 bits (de 0000 à 1111) et indiquer à quel nombre décimal elles correspondent si on les interprète en complément à 2.

Pour convertir un nombre négatif binaire en sa valeur décimale, on doit d'abord inverser tous les bits (les 0 deviennent 1 et les 1 deviennent 0), ensuite on ajoute 1 au résultat obtenu et le résultat donne la valeur absolue du nombre, qu'on rend négative.

Binaire	Valeur décimale (complément à 2)
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

Ainsi, avec 4 bits, on peut représenter en complément à deux les nombres allant de -8 à +7.

Exercice 3.14 (A Rendre)

Étant donné deux nombres représentés en complément à 2, comment peut-on savoir lequel est le plus grand en n'utilisant que des comparaisons de bits entre eux ?

Écrire un programme dans le langage de votre choix pour effectuer cette comparaison. (Avec ou sans saisie de ces nombres par l'utilisateur selon vos connaissances dans le langage choisi)

Pour vous aider : Si les bits de signe sont différents, c'est le nombre où il vaut 0 qui est le plus grand...

Réponse :

On cherche à déterminer lequel de deux nombres représentés en complément à 2 est le plus grand, uniquement en comparant leurs bits. En complément à 2, le bit le plus à gauche, appelé bit de signe, indique si le nombre est positif (0) ou négatif (1).

Le programme commence par convertir les deux nombres en binaire sur un nombre suffisant de bits afin d'obtenir leur représentation complète. Cela permet de gérer correctement à la fois les nombres positifs et négatifs. Ensuite, il récupère le bit de signe de chaque nombre.

Si les bits de signe sont différents, le nombre dont le bit de signe vaut 0 est forcément le plus grand, car il est positif. Si les deux nombres ont le même signe, le programme compare les bits restants un par un, de gauche à droite. Pour les nombres positifs, un bit 1 est plus grand qu'un bit 0. Pour les nombres négatifs, c'est l'inverse : plus la valeur binaire est grande, plus le nombre réel est petit.

Enfin, si tous les bits sont identiques, les deux nombres sont égaux.

Au début, j'ai rencontré des difficultés liées au nombre de bits à utiliser, car tous les nombres ne peuvent pas être représentés sur seulement 2 bits par exemple. J'ai également dû chercher comment gérer correctement les nombres négatifs pour éviter que le programme affiche des résultats incohérents, comme dire que -3 est plus grand que -2.

Au final, j'ai fait l'inverse de ce que je faisais au départ. Avant, je demandais d'abord à l'utilisateur le nombre de bits, puis les nombres a et b. Maintenant, je demande directement les nombres a et b, et ensuite mon programme calcule automatiquement le nombre minimum de bits nécessaires pour représenter correctement ces deux nombres. Pour cela, j'ai utilisé les fonctions `max` et `abs`, que j'ai apprises l'année dernière dans mes cours de NSI (Numérique et Sciences Informatiques) en terminale.

Voici le programme (également disponible dans l'annexe voir fichier README) :

```
1 # Saisie des nombres en decimal
2 a = int(input("Entrez le premier nombre : "))
3 b = int(input("Entrez le deuxieme nombre : "))
4
5 # Calcul du nombre minimum de bits pour représenter les deux nombres
6 max_val = max(abs(a), abs(b))
7 bits = 1
8 while (1 << (bits - 1)) <= max_val:
9     bits += 1
10
11 print(f"Nombre de bits utilise : {bits}")
12
13 # Fonction pour convertir un nombre en complement a 2 sur 'bits' bits
14 def to_complement2(n, bits):
15     if n < 0:
16         n = (1 << bits) + n # ajouter 2^bits si le nombre est negatif
17     return n
18
19 # Conversion des nombres en complement a 2
20 a_bin = to_complement2(a, bits)
```

```

21 b_bin = to_complement2(b, bits)
22
23 # Determiner le bit de signe (0 = positif, 1 = negatif)
24 def bit_signe(n, bits):
25     if n >= (1 << (bits - 1)):
26         return 1
27     else:
28         return 0
29
30 signe_a = bit_signe(a_bin, bits)
31 signe_b = bit_signe(b_bin, bits)
32
33 # Comparaison selon le signe
34 if signe_a != signe_b:
35     if signe_a == 0:
36         print(f"{a} est plus grand que {b}")
37     else:
38         print(f"{b} est plus grand que {a}")
39 else:
40     # Meme signe : comparer les valeurs en complement a 2
41     if a_bin > b_bin:
42         print(f"{a} est plus grand que {b}")
43     elif a_bin < b_bin:
44         print(f"{b} est plus grand que {a}")
45     else:
46         print(f"{a} et {b} sont egaux")

```

Voici quelques tests pour verifier que le programme fonctionne correctement :

Test avec a et b positifs, et $b > a$:

```

>>> %Run 'chap 3.py'

Entrez le premier nombre : 3
Entrez le deuxieme nombre : 6
Nombre de bits utilise : 4
6 est plus grand que 3

```

Test avec a et b positifs, et $a > b$:

```

>>> %Run 'chap 3.py'

Entrez le premier nombre : 6
Entrez le deuxieme nombre : 3
Nombre de bits utilise : 4
6 est plus grand que 3

```

Test avec a negatif et b positif :

```

>>> %Run 'chap 3.py'

Entrez le premier nombre : -2
Entrez le deuxieme nombre : 9
Nombre de bits utilise : 5
9 est plus grand que -2

```


Test avec a positif et b negatif :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 100
Entrez le deuxieme nombre : -4
Nombre de bits utilise : 8
100 est plus grand que -4
```

Test avec a et b negatifs :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : -4
Entrez le deuxieme nombre : -93
Nombre de bits utilise : 8
-4 est plus grand que -93
```

Test avec a et b egaux :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 94
Entrez le deuxieme nombre : 94
Nombre de bits utilise : 8
94 et 94 sont egaux
```

Exercice 3.15 (Bonus)

Identifier les deux valeurs pour lesquelles le complément à deux ne change pas le bit de signe de la représentation d'un nombre.

Il est important de ne pas oublier d'expliquer et de justifier votre réponse.

Pour vous aider : relire l'encadré "Important" du chapitre.

Réponse :

En complément à 2, le bit de signe (le bit le plus à gauche) indique si un nombre est positif (0) ou négatif (1). D'après l'encadré "Important" du chapitre, toutes les opérations en complément à 2 se font sur un nombre défini de bits, comme 8 bits, 16 bits, etc. Pour que le complément à 2 ne change pas le bit de signe, il faut que le nombre soit 0, soit -1.

En effet, pour 0, sa représentation binaire sur n bits est composée uniquement de 0 et son complément à 2 reste 0, donc le bit de signe reste 0.

Pour mieux comprendre, prenons un exemple : pour 0 sur 4 bits (0000), on fait son complément à deux, c'est-à-dire qu'on inverse tous les bits (1111) puis on ajoute 1 (0000). Donc le bit de signe reste 0.

Pour -1, sa représentation binaire est composée uniquement de 1 et son complément à 2 (qui consiste à inverser tous les bits puis ajouter 1) redonne -1, donc le bit de signe reste 1.

Par exemple, pour -1 sur 4 bits (1111), on fait son complément à deux, on inverse tous les bits (0000), puis on ajoute 1 (1111). Donc le bit de signe reste 1.

Ainsi, les deux valeurs pour lesquelles le complément à 2 ne modifie pas le bit de signe sont 0 et -1, et cette règle est valable quel que soit le nombre de bits utilisé.

Sources

- Wikipédia — *Système négabinaire* : https://fr.wikipedia.org/wiki/Syst%C3%A8me_n%C3%A9gabinaire
- Cours de NSI (Terminale) — notions sur le complément à 2 et les comparaisons en binaire.

Chapitre 4 — Représentation des Nombres Flottants et des Caractères

4.2 Une Première Série d'Exercices

Exercice 4.3 (A Rendre)

Une légende dit qu'en guise de récompense, l'inventeur (indien) du jeu d'échecs demande un grain de blé sur la première case de l'échiquier, deux grains sur la seconde, quatre sur la troisième et ainsi de suite jusqu'à la soixante-quatrième.

1. Approximer en base 10 le nombre de grains à poser sur la dernière case.
2. En supposant qu'il faut dix grains de blé pour faire un gramme, approximer le poids que cela représente. (Convertir le résultat en tonnes.)

Réponse :

Les grains de riz évoluent de cette manière :

- 1 grain sur la première case,
- 2 grains sur la deuxième case,
- 4 grains sur la troisième case,
- et ainsi de suite jusqu'à la 64^e case (puisque sur un jeu d'échecs on a 64 cases).

On remarque que chaque case contient le double de la case précédente. On reconnaît ici une suite géométrique de raison 2, où le premier terme est 1.

On cherche le nombre de grains sur la dernière case.

Pour une suite géométrique, le n -ième terme u_n est donné par la formule :

$$u_n = u_1 \times r^{n-1}$$

Dans notre cas :

- $u_1 = 1$ (le premier terme),
- $r = 2$ (la raison),
- $n = 64$ (la dernière case).

Donc le nombre de grains sur la dernière case est :

$$u_{64} = 1 \times 2^{63} = 2^{63}$$

On peut l'approximer en base 10. On sait que :

$$2^{10} \approx 10^3$$

Donc :

$$2^{60} = (2^{10})^6 \approx (10^3)^6 = 10^{18}$$

Puis :

$$2^{63} = 2^3 \times 2^{60} = 8 \times 10^{18} \approx 8 \cdot 10^{18} \text{ grains.}$$

Donc sur la dernière case, il y a environ 8 milliards de milliards de grains.

Il faut 10 grains pour faire 1 gramme.

— Poids en grammes :

$$\text{poids (g)} = \frac{2^{63}}{10} \approx 8 \times 10^{17} \text{ g}$$

— Convertir en kilogrammes : $1 \text{ kg} = 1000 \text{ g}$

$$\text{poids (kg)} = \frac{8 \times 10^{17}}{10^3} = 8 \times 10^{14} \text{ kg}$$

— Convertir en tonnes : $1 \text{ tonne} = 1000 \text{ kg}$

$$\text{poids (tonnes)} = \frac{8 \times 10^{14}}{10^3} = 8 \times 10^{11} \text{ tonnes}$$

Donc :

- Le nombre de grains sur la dernière case est environ 8×10^{18} .
- Le poids correspondant est environ 8×10^{11} tonnes.

Exercice 4.4 (Au Choix)

- (1) On estime le nombre de parties possibles aux Échecs à quelque chose comme 10^{123} .
- (2) Le nombre de positions possibles au jeu de Go est quelque chose comme 10^{171} .
- (3) Le nombre de parties possibles au Backgammon est quelque chose comme 10^{144} .

Combien faut-il de bits pour représenter chacun de ces nombres (1)(2)(3) ?

Pour vous aider : Exercice applicatif (voir l'exemple du cours).

Réponse :

Comme je bloquais un peu sur cet exercice, j'ai cherché un peu sur internet et je suis tombé sur cette conversation : <https://openclassrooms.com/forum/sujet/combien-de-bits-pour-un-nombre-decimal>, c'est grâce à cela que j'ai découvert la formule. Je ne vais pas utiliser une formule directement sans savoir si elle est vraie, mais cela m'a débloqué pour faire cet exercice.

On sait qu'un bit ne peut stocker que deux valeurs : 0 ou 1. En général, avec b bits, on peut représenter 2^b valeurs différentes, allant de 0 à $2^b - 1$.

Pour pouvoir stocker un nombre N , il faut que le nombre de valeurs représentables par b bits soit au moins égal à N . Autrement dit, il faut que :

$$2^b \geq N$$

Pour trouver b à partir de N , on utilise le logarithme en base 2 :

$$b \geq \log_2(N)$$

Comme b doit être un nombre entier, on prend le plus petit entier supérieur ou égal à $\log_2(N)$. C'est pour cela qu'on peut écrire :

$$b = \lceil \log_2(N) \rceil$$

On nous donne trois nombres très grands :

- 10^{123} pour les parties possibles aux Échecs
- 10^{171} pour les positions possibles au jeu de Go
- 10^{144} pour les parties possibles au Backgammon

Pour trouver le nombre de bits nécessaires, on utilise la formule qu'on a trouvée, mais on peut d'abord convertir le logarithme en base 2 à l'aide de la formule :

$$\log_2(N) = \frac{\log_{10}(N)}{\log_{10}(2)}$$

et on sait que $\log_{10}(2) \approx 0.301$.

Étape 1 : pour 10^{123}

$$\log_2(10^{123}) = 123 \times \log_2(10) = \frac{123}{0.301} \approx 408.64$$

Nombre de bits nécessaires :

$$b_1 = \lceil 408.64 \rceil = 409 \text{ bits}$$

Étape 2 : pour 10^{171}

$$\log_2(10^{171}) = 171 \times \log_2(10) = \frac{171}{0.301} \approx 567.77$$

Nombre de bits nécessaires :

$$b_2 = \lceil 567.77 \rceil = 568 \text{ bits}$$

Étape 3 : pour 10^{144}

$$\log_2(10^{144}) = 144 \times \log_2(10) = \frac{144}{0.301} \approx 478.40$$

Nombre de bits nécessaires :

$$b_3 = \lceil 478.40 \rceil = 479 \text{ bits}$$

Conclusion :

- Pour le nombre de parties aux Échecs (10^{123}) : 409 bits
- Pour le nombre de positions au Go (10^{171}) : 568 bits
- Pour le nombre de parties au Backgammon (10^{144}) : 479 bits

Exercice 4.6 (Bonus)

Si nous nous référons au programme vu au chapitre précédent et rappelé ici :

1. Évaluer l'ordre de grandeur d'un nombre de tours de boucles qu'il effectue. En considérant ici que chaque tour de boucle fait trois opérations : (comparer avec 0, ajouter 1, recommencer la boucle) et que chaque opération prend un tic d'horloge (la fréquence de votre processeur indique le nombre de tics d'horloge par seconde).
2. Évaluer l'ordre de grandeur du temps nécessaire pour exécuter le programme. Placer ensuite ce code du programme (par copier-coller) dans un fichier nommé `foo.c`, le compiler avec la ligne de commande `gcc foo.c`, mesurer son temps d'exécution avec la ligne de commande `time a.out`.
3. Comparer le temps mesuré avec l'estimation "évaluée" de (2). Donnez quelques éléments de justification (c'est le plus important).

Pour vous aider : Quelques recherches à effectuer sur internet vous donneront quelques éléments de réponses...

Rappel du programme :

```
1 int main() {  
2     int t = 1;  
3     while (t != 0) {  
4         t = t + 1;  
5     }  
6     return 0;  
7 }
```

Réponse :

1 : Évaluer l'ordre de grandeur du nombre de tours de boucle

On initialise t à 1. La boucle continue tant que $t \neq 0$. À chaque tour, on ajoute 1 à t .

Ainsi :

- t est un entier de type `int`, souvent représenté sur 32 bits.
- La valeur maximale pour un entier 32 bits est $2^{31} - 1 = 2\,147\,483\,647$.
- La boucle effectue donc environ 2×10^9 tours avant de s'arrêter.

Nombre d'opérations par tour :

1. Comparer $t \neq 0$
2. Ajouter 1 à t
3. Recommencer la boucle

Donc pour 2×10^9 tours, le programme réalise environ 6×10^9 opérations au total.

2 : Estimation du temps

Chaque opération prend approximativement un tic d'horloge. Le processeur utilisé fonctionne à **3.6 GHz**, soit 3.6×10^9 tics par seconde.

Processeur 12th Gen Intel(R) Core(TM) i7-12700KF (3.60 GHz)

FIGURE 9 – Fréquence du processeur utilisée pour l'estimation (3.6 GHz).

Le temps d'exécution est donc :

$$t = \frac{6 \times 10^9}{3.6 \times 10^9} \approx 1,67 \text{ secondes}$$

Ainsi, le programme met environ **1,7 seconde** à s'exécuter.

3 : Comparer avec le temps mesuré

On vérifie ce résultat :

```
real 0m1.679s  
user 0m1.679s  
sys 0m0.000s
```

Pour répondre à cette question, j'ai utilisé plusieurs sources :

- <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/> (même si elle ne parle pas de code "pur", je trouve que les exemples utilisés illustrent bien le problème)
- <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/> (pour tout ce qui est complexité/nombre d'opérations réelles, temps d'accès, etc.)
- <https://cahier-de-prepa.fr/psi-michelet/download?id=239> (pour la complexité également)

Le temps mesuré pour exécuter le programme est légèrement différent de l'estimation théorique pour plusieurs raisons. Tout d'abord, l'estimation ne prend en compte que le nombre d'opérations de base et la fréquence du processeur, en supposant que chaque opération dure exactement un tic d'horloge. En vérité, le processeur réalise également d'autres opérations telles que la gestion du cache, des interruptions et des processus de fond, ce qui peut avoir un impact sur le temps global. Par ailleurs, notre programme comporte une boucle qui augmente de manière linéaire avec la variable t , et la complexité du programme est directement liée à la dimension du type entier utilisé (en l'occurrence 32 bits). Si nous avions opté pour un nombre plus élevé, la durée aurait été considérablement plus longue.

On peut aussi comparer cette situation à l'estimation du temps dans un projet réel. Une société qui missionne un développeur a besoin de connaître le coût d'une tâche pour faire un devis, mais le temps estimé ne reflète jamais tout le travail réel : coder, réfléchir, organiser, se tromper, recommencer ou se former sont des éléments souvent sous-estimés ou oubliés. De plus, l'estimation varie selon la personne qui réalise la tâche : ce qui prend une journée à un développeur peut en prendre deux à un autre, ou moins à un autre encore. Ainsi, l'estimation n'est jamais parfaitement objective.

Finalement, certains logiciels ou systèmes d'exploitation peuvent occasionner de légères variations dans l'évaluation du temps réel, et la façon dont l'ordinateur accède à la mémoire ou aux registres peut également affecter le temps écoulé. Ainsi, bien que l'évaluation fournisse une approximation correcte, la durée réelle peut varier légèrement en raison de ces éléments matériels, logiciels et humains, ce qui est à la fois normal et prévisible.

Conclusion

- La complexité du programme est linéaire pour un entier sur 32 bits.
- Le programme effectue environ 2×10^9 tours et 6×10^9 opérations.
- Avec un processeur à 3.6 GHz, le temps d'exécution théorique est d'environ 1,7 seconde.
- Le temps mesuré en pratique peut changer, mais il sera toujours très proche (des 1,7 secondes dans notre cas).

4.10 Une Seconde Série d'Exercices

Vous devez envoyer, pour valider ce quatrième chapitre, les exercices **4.3 (A Rendre)**, **4.13 (A Rendre)** et **un exercice (Au Choix)**. Pour celles et ceux qui souhaitent un petit plus : le **4.6** est pour un Bonus, l'exercice supplémentaire **4.17** ajoutera un second Bonus.

Exercice 4.13 (A Rendre)

Écrire un programme dans le langage de votre choix pour multiplier deux nombres représentés en base 10 et en virgule flottante entrés par l'utilisateur.

Penser à normaliser le résultat de façon à n'avoir qu'un seul chiffre à gauche de la virgule dans le résultat.

Réponse :

```
1 # Exercice 4.13
2
3 # Donner deux nombres flottants a et b
4 a = float(input("Entrez le premier nombre : "))
5 b = float(input("Entrez le deuxieme nombre : "))
6
7 # Multiplier les deux nombres
8 resultat = a * b
9
10 # Initialiser l'exposant pour apres
11 expo = 0
12
13 # Normaliser pour avoir exactement un chiffre a gauche de la virgule
14 if resultat != 0:
15     while abs(resultat) >= 10:
16         resultat = resultat/10
17         expo = expo + 1
18     while abs(resultat) < 1:
19         resultat = resultat * 10
20         expo = expo - 1
21
22 # Affichage du resultat normalise
23 print("Resultat normalise :", resultat)
24 print("Exposant :", expo)
25 print("Resultat en notation scientifique :", str(resultat) + " x 10^" + str(expo))
```

Test pour vérifier que tout fonctionne bien (code en annexe) :

```
>>> %Run 'chap 4.py'
Entrez le premier nombre : 434453576.7
Entrez le deuxième nombre : 4.5
Résultat normalisé : 1.95504109515
Exposant : 9
Résultat en notation scientifique : 1.95504109515 x 10^9

>>> %Run 'chap 4.py'
Entrez le premier nombre : 434453576.7
Entrez le deuxième nombre : 4.5
Résultat normalisé : 1.95504109515
Exposant : 9
Résultat en notation scientifique : 1.95504109515 x 10^9
```


Exercice 4.17 (Bonus)

Écrire dans le langage de votre choix un programme qui calcule la fonction exponentielle selon la formule de Taylor. Constaté l'erreur d'approximation produite.

Rappel de la formule :

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots + \frac{x^n}{n!}$$

Réponse :

```
1 # Entrer la valeur de x
2 x = float(input("Entrez la valeur de x : "))
3
4 # Entrer le nombre de termes n pour approximer la somme
5 n = int(input("Entrez le nombre de termes pour l'approximation (plus grand = plus
   precis) : "))
6
7 # Initialisation de la somme avec le premier terme (1)
8 exp = 1.0
9
10 # Boucle pour calculer les termes suivants
11 fact = 1      # Facteur pour le calcul de la factorielle
12 puissance = 1 # x^i
13
14 for i in range(1, n+1):
15     puissance = puissance * x    # Calcul de x^i
16     fact = fact * i              # Calcul de i!
17     terme = puissance / fact    # Terme de la serie
18     exp = terme + exp           # Ajouter le terme a la somme
19
20 # Affichage du resultat approximatif
21 print("Approximation de e^", x, "avec", n, "termes : ", exp)
```

Test pour verifier que le programme fonctionne :

The image shows a terminal window with the following output:

```
>>> %Run 'chap 4 17.py'
Entrez la valeur de x : 8
Entrez le nombre de termes pour l'approximation (plus grand = plus précis) : 6
Approximation de e^ 8.0 avec 6 termes : 934.1555555555556
```

Below the terminal is a digital calculator interface. The display shows the result of e^8 as 2980,95799. The calculator has a grid of buttons for various mathematical operations and constants.

Plus x est grand et/ou n est petit, plus l'erreur sera enorme (par exemple avec $x = 656$ et $n = 4$) :

```
>>> %Run 'chap 4 17.py'
Entrez la valeur de x : 656
Entrez le nombre de termes pour l'approximation (plus grand = plus précis) : 4
Approximation de e^ 656.0 avec 4 termes : 7763477265.0
```

7,89187374E284

Inv	Deg	x!	(	)	C	⌫
ln	sin	%	7	8	9	÷
log	cos	√	4	5	6	×
^	tan	1/x	1	2	3	-
Exp	π	e	0	,	=	+

Lorsque la valeur de x augmente les termes de la série de Taylor deviennent plus importants ce qui demande une précision et donc davantage de chiffres après la virgule.

Sources

- OpenClassrooms (Forum) — *Combien de bits pour un nombre décimal ?* : <https://openclassrooms.com/forum/sujet/combien-de-bits-pour-un-nombre-decimal>
- Apprendre-la-programmation.net — estimation du temps : <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/>
- Interstices.info — article sur le temps et les sciences de l'information : <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/>
- Cahier-de-prepa.fr — document sur la complexité : <https://cahier-de-prepa.fr/psi-michelet/download?id=239>
- Cours de NSI (Terminale) — suites géométriques, logarithmes, complexité, approximation de Taylor.

Chapitre 5 - Algèbre de Boole

5.4 Exercices

Pour ce cinquième chapitre, les exercices à compléter et à rendre dans le PDF sont : **5.10** (quatre équations au choix), **5.11** (deux tables au choix), un exercice "**Au choix**" et **5.9** pour un **Bonus** (Attention de bien respecter la consigne).

Exercice 5.5 (Au Choix)

À l'aide d'une table de vérité, vérifier si le **NON-ET** est associatif, c'est-à-dire si :

$$\overline{\overline{a \cdot b} \cdot c} = \overline{a \cdot \overline{b \cdot c}}$$

Réponse :

Variables et valeurs possibles :

- Variables : a, b, c
- Valeurs possibles : V (vrai), F (faux)
- Nombre de lignes dans la table : $2^3 = 8$

Colonnes intermédiaires

On crée des colonnes intermédiaires :

1. a, b, c (valeurs de base)
2. $a \cdot b$
3. non-et $a \cdot b$
4. (non-et $a \cdot b$) $\cdot c$
5. Résultat gauche : $\overline{\overline{a \cdot b} \cdot c}$
6. $b \cdot c$
7. non-et $b \cdot c$
8. $a \cdot$ (non-et $b \cdot c$) $\overline{\overline{a \cdot b \cdot c}}$
9. Résultat droite : $\overline{\overline{a \cdot b \cdot c}}$
10. Comparaison si gauche = droite

Table de vérité

a	b	c	a·b	non-et a·b	(non-et a·b)·c	Résultat gauche	b·c	non-et b·c	a·(non-et b·c)	Résultat droite
F	F	F	F	V	F	V	F	V	F	V
F	F	V	F	V	V	F	F	V	F	V
F	V	F	F	V	F	V	F	V	F	V
F	V	V	F	V	V	F	V	F	F	V
V	F	F	F	V	F	V	F	V	V	F
V	F	V	F	V	V	F	F	V	V	F
V	V	F	V	F	F	V	F	V	V	F
V	V	V	V	F	F	V	V	F	V	F

On peut regarder chaque lignes :

- Lignes 1, 3, 6 : les résultat gauche sont égal au résultat droite donc vrai.
- Toutes les autres lignes, les résultat gauche sont différent du résultat droite donc faux.

Conclusion

Comme toutes les lignes ne donnent pas le même résultat à gauche et à droite, **le NON-ET n'est pas associatif**.

Exercice 5.9 (Bonus)

Imaginons que dans notre langage de tous les jours, nous utilisons l'algèbre de Boole. Que répondrait alors la sage-femme lorsque le nouveau père lui demande : « Alors, c'est un garçon ou une fille ? ».

Dans quel cas la réponse de l'infirmière serait-elle différente si le père lui demandait : « C'est un garçon ou exclusif une fille ? ».

Il est important d'utiliser ici l'ontologie (les mots-métiers) de la sage-femme et non celle du logisticien. La réponse doit être donnée sous forme de phrase naturelle, et non avec une expression logique. Il faut bien expliquer et développer vos réponses.

Réponse :

Quand le papa demande à la sage-femme : "Alors, c'est un garçon ou une fille ?", dans le langage de tous les jours, le mot "ou" veut dire "soit l'un, soit l'autre, mais pas les deux". La sage-femme sait donc instinctivement que le bébé ne peut être ni un garçon ni une fille en même temps et elle répondra simplement : "C'est un garçon" ou "C'est une fille".

En logique, ce "ou" correspond à ce qu'on appelle le "ou exclusif". Si on prenait le "ou" au sens logique inclusif, cela voudrait dire "un garçon ou une fille ou les deux", ce qui n'aurait pas de sens pour un seul bébé, sauf s'il s'agit de jumeaux de sexes différents. Dans ce cas précis, la réponse serait différente : on pourrait alors dire qu'il y a à la fois un garçon et une fille.

Ainsi, dans notre quotidien, on utilise sans s'en rendre compte le "ou exclusif", et ce n'est que dans des cas particuliers comme une naissance de jumeaux que le sens logique du mot "ou" (inclusif) pourrait modifier la réponse.

Exercice 5.10 (À rendre)

Simplifier algébriquement les expressions algébriques de gauche, jusqu'à obtenir l'expression de droite.

Choisissez quatre équations ci-dessous selon le degré de difficulté souhaité. Gardez bien l'ordre alphabétique des équations choisies. Par exemple : (a, c, g, i).

b.

$$f = (a \cdot c + b \cdot \bar{c}) \cdot (\bar{a} + \bar{c}) \cdot b$$

Étapes de simplification :

$$\begin{aligned} (a \cdot c + b \cdot \bar{c}) \cdot (\bar{a} + \bar{c}) &= a \cdot c \cdot \bar{a} + a \cdot c \cdot \bar{c} + b \cdot \bar{c} \cdot \bar{a} + b \cdot \bar{c} \cdot \bar{c} && \text{(distribution)} \\ &= 0 + 0 + b \cdot \bar{c} \cdot \bar{a} + b \cdot \bar{c} && \text{(car } a \cdot \bar{a} = 0 \text{ et } c \cdot \bar{c} = 0) \\ &= b \cdot \bar{c} \cdot (\bar{a} + 1) && \text{(factorisation)} \\ &= b \cdot \bar{c} && \text{(puisque } \bar{a} + 1 = 1) \end{aligned}$$

c.

$$f = a \cdot \bar{b} \cdot \bar{c} + a \cdot b \cdot \bar{c} + a \cdot b \cdot c$$

Étapes de simplification :

$$\begin{aligned} f &= a \cdot (\bar{b} \cdot \bar{c} + b \cdot \bar{c} + b \cdot c) && \text{(factorisation par } a) \\ &= a \cdot ((\bar{b} + b) \cdot \bar{c} + b \cdot c) && \text{(factorisation par } \bar{c} \text{ dans les deux premiers termes)} \\ &= a \cdot (\bar{c} + b \cdot c) && \text{(car } \bar{b} + b = 1) \\ &= a \cdot (b + \bar{c}) && \text{(réécriture pour simplifier l'expression)} \end{aligned}$$

e.

$$f = (a + b) \cdot (\bar{a} + b)$$

Étapes de simplification :

$$\begin{aligned} f &= a \cdot \bar{a} + a \cdot b + b \cdot \bar{a} + b \cdot b && \text{(distribution)} \\ &= 0 + a \cdot b + b \cdot \bar{a} + b && \text{(car } a \cdot \bar{a} = 0 \text{ et } b \cdot b = b) \\ &= a \cdot b + b \cdot (\bar{a} + 1) && \text{(factorisation par } b) \\ &= a \cdot b + b && \text{(car } \bar{a} + 1 = 1) \\ &= b && \text{(absorption : } a \cdot b + b = b) \end{aligned}$$

f.

$$f = (x \cdot \bar{y} + z) \cdot (x + \bar{y}) \cdot z$$

Étapes de simplification :

$$\begin{aligned} (x \cdot \bar{y} + z) \cdot (x + \bar{y}) &= x \cdot \bar{y} \cdot x + x \cdot \bar{y} \cdot \bar{y} + z \cdot x + z \cdot \bar{y} && \text{(distribution)} \\ &= x \cdot \bar{y} + 0 + z \cdot x + z \cdot \bar{y} && \text{(car } x \cdot x = x, \bar{y} \cdot \bar{y} = \bar{y}) \\ &= x \cdot \bar{y} + z \cdot (x + \bar{y}) && \text{(factorisation par } z) \\ f &= (x \cdot \bar{y} + z \cdot (x + \bar{y})) \cdot z \\ &= (x + \bar{y}) \cdot z && \text{(simplification finale)} \end{aligned}$$

Exercice 5.11 (À rendre)

Utilisez les tables de Karnaugh suivantes pour obtenir la forme algébrique simplifiée de la fonction qu'elles décrivent.

Choisissez deux tables ci-dessous parmi les quatre. Comme pour l'exercice précédent, gardez bien l'ordre alphabétique des tables choisies. Par exemple : (b, d).

Table (a)

La table (a) est :

$cd \backslash ab$	FF	FV	VV	VF
FF	V	V	V	V
FV	V			V
VV	V			V
VF	V	V	V	V

On remarque que toutes les cases de la colonne FF et de la colonne VF contiennent V sur toutes les lignes. Cela signifie que la fonction est vraie pour toutes les valeurs de a et c , et ces V peuvent être couverts par le terme le plus simple qui inclut b et d . Ainsi, la fonction simplifiée s'écrit :

$$f_a(a, b, c, d) = b + d$$

Table (c)

La table (c) est :

$cd \backslash ab$	FF	FV	VV	VF
FF		V	V	
FV	V	V		
VV	V	V	V	V
VF		V	V	

Pour cette table, on peut former plusieurs rectangles de V . Le carré central 2×2 couvrant les colonnes FV et VV ainsi que les lignes FV et VV correspond à $c.d$. Les V de la colonne FV peuvent être regroupés verticalement et correspondent à $b.c$. Pareil pour les V de la colonne VV peuvent être regroupés verticalement et correspondent à $b.d$. En combinant ces termes, on obtient l'expression simplifiée :

$$f_c(a, b, c, d) = c.d + b.c + b.d$$

Sources de tout les chapitres

- Python-3.com — explication de la méthode `.upper()` en Python : <https://fr.python-3.com/?p=2788>
- BornToCode.fr — insertion et personnalisation de code source en LaTeX : <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>
- WolframAlpha — outil de vérification de calculs et conversions : <https://www.wolframalpha.com/>
- Wikipédia — discussion sur la numération romaine : https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine
- Wikipédia — système négabinaire : https://fr.wikipedia.org/wiki/Syst%C3%A8me_n%C3%A9gabinaire
- OpenClassrooms (Forum) — nombre de bits nécessaires pour représenter un nombre décimal : <https://openclassrooms.com/forum/sujet/combien-de-bits-pour-un-nombre-decimal>
- Apprendre-la-programmation.net — estimation du temps d'exécution en programmation : <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/>
- Interstices.info — article sur le temps, la complexité et les systèmes informatiques : <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/>
- Cahier de prépa (PSI Michelet) — notions de complexité algorithmique : <https://cahier-de-prepa.fr/psi-michelet/download?id=239>
- Cours de NSI (Terminale) — représentation des nombres (binaires, complémentaires à 2, flottants), suites, logarithmes, complexité algorithmique et algèbre de Boole.